

FORMATO DE ENTREGA DE EVIDENCIAS

Serie de guías · Desarrollo de APIs con FastAPI

1. Datos del aprendiz

Nombre completo	Joel Andrés Mendoza Buriticá
Número de documento	1042853355
Programa	Análisis y Desarrollo de Software (ADSO)
Ficha	3169892
Número de la guía	Guía 03
Nombre de la guía	Desarrollar los servicios del back-end del sistema aplicando un framework moderno de Python
Instructor	Mateo Arroyave
Fecha de entrega	06 / 08 / 2026

Cómo diligenciar este formato

Complete cada evidencia reemplazando el texto en gris. Escriba dentro de los recuadros y pegue las capturas donde se indica. Al terminar, exporte a PDF (Archivo ▶ Guardar como ▶ PDF) y suba el archivo a Google Classroom en la actividad correspondiente.

2. Enlace al repositorio de GitHub

Muchas evidencias piden el código en GitHub. Pegue aquí el enlace de su repositorio (debe ser público):

<https://github.com/JBUR10/APIS-FASTAPIS.git>

3. Evidencia 1 — Reflexión inicial

Código de la evidencia	GA2-AA1-EV01
Tipo	Conocimiento

Respuestas a las preguntas de reflexión:

1. Si publicara esa API en internet tal como está, ¿qué cosas malas podría hacer un desconocido con los endpoints POST, PUT y DELETE?

Ese desconocido podría inventar datos, modificar informaciones gracias a los PUT y borrar registros sin problema alguno por lo que se consideraría una afectación a la integridad del sistema. Además, es posible que estos datos manipulados por el desconocido sean irre recuperables, corruptos o directamente cause un fallo en la app.

2. Cuando entra a una red social, primero pone su usuario y contraseña (una vez) y luego navega sin volver a escribirla en cada clic. ¿Cómo cree que el sistema “recuerda” que es usted en cada acción?

Cuando se realiza un back-end necesitamos que la sesión sea segura por lo que hacemos uso de un token que permite que el sistema “recuerde” quienes somos y si efectivamente seguimos siéndolo, también nos permite que los cierres de sesión sean correctos ya que normalmente sin el uso de estos la sesión puede quedar iniciada.

3. ¿Qué diferencia hay entre demostrar quién es usted y tener permiso para hacer algo? Piense en un edificio: entrar con su cédula vs. tener llave de la oficina del jefe.

Lo primero es una identificación, es como decirle al sistema soy Joel y estoy en el sistema, el permiso es poder hacer algo dentro de ese mismo sistema, ese conjunto de acciones que yo, Joel, como usuario tengo permitido hacer, usando el ejemplo al entrar puedo mostrar mi identificación al vigilante y después de ello dirigirme a la oficina del jefe a realizar ciertas acciones que, por tener la llave otros no pueden hacer.

4. Si una empresa guardara las contraseñas de sus usuarios tal cual, en una tabla, ¿qué pasaría si alguien roba esa tabla? ¿Cómo se podría evitar el daño?

Si un atacante logra acceder la base de datos, tendría acceso inmediato a todas las contraseñas de los usuarios ya que estas se encuentran tal cual, en texto plano, por lo que podría simplemente ingresar a sus cuentas o incluso vender esa información, normalmente se evita hasheando las contraseñas, esto formatea este texto a un valor irreversible.

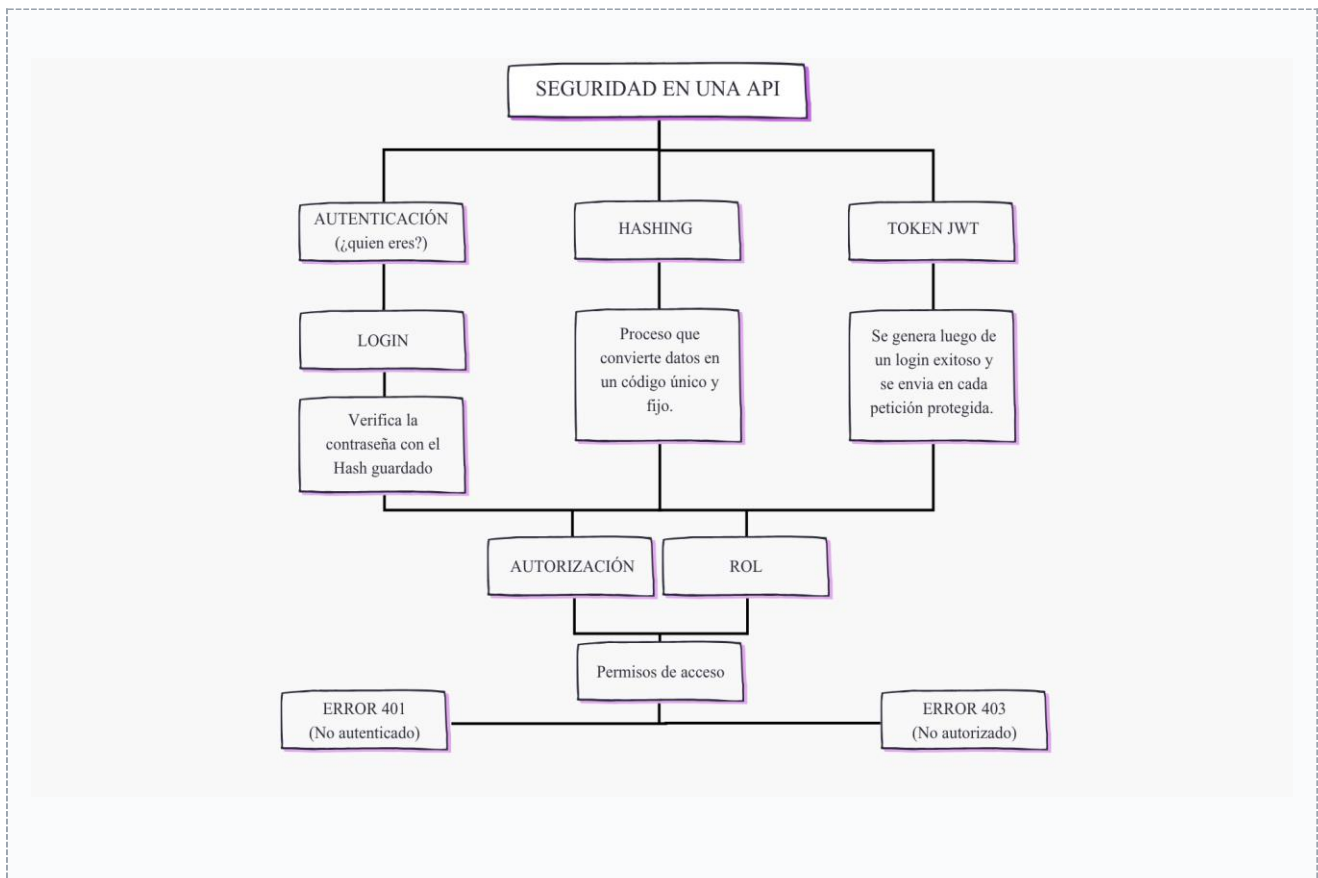
4. Evidencia 2 — Contextualización

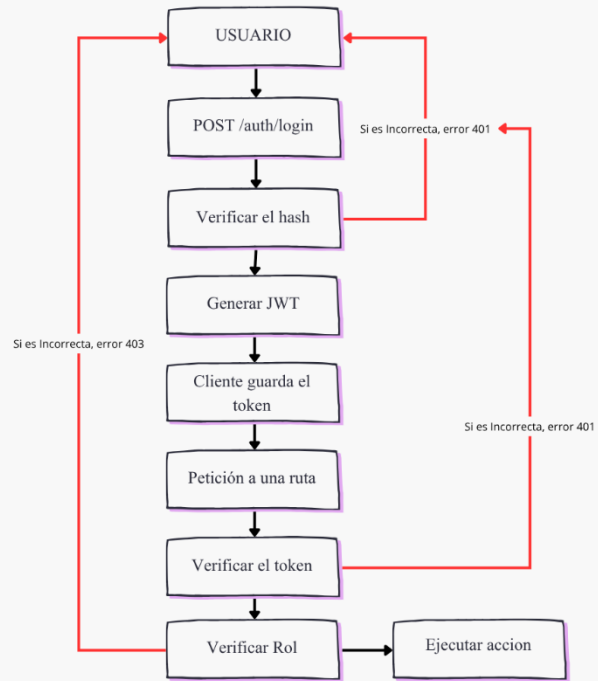
Código de la evidencia	GA2-AA1-EV02
Tipo	Conocimiento

Cuadro comparativo / mapa conceptual solicitado en la guía. Puede escribirlo en el recuadro o pegar una imagen de su cuadro/mapa:

Mapa conceptual que relacione: autenticación, autorización, hashing, token JWT, login, rol, 401 y 403.

Diagrama del flujo (login → token → petición protegida).





5. Evidencia 3 — Apropiación (laboratorio)

Código de la evidencia	GA2-AA2-EV03
Tipo	Desempeño

Descripción de lo que hizo: explique brevemente los pasos que siguió y los ejercicios que resolvió.

1. Cree la autenticación en base a los criterios dados por la guía, estos fueron implementados en todos mis endpoints (exceptuando los públicos). Esto usando un usuario y contraseña, esta última hashheada usando bcrypt, implemente el registro de usuarios que usa la función `def hashear_password(password: str) -> str:`
`hasheado = bcrypt.hashpw(password.encode(), bcrypt.gensalt())`
`return hasheado.decode()` para hashear la contraseña del usuario, en la función en la que registramos al usuario informe claramente que su rol es el de cliente.

Capturas de pantalla: pegue la evidencia de su API funcionando en /docs y de los endpoints probados.

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/productos' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbIiImV4cCI6MTc4NjAzNTE1MH0.Ov1FjT9iIGqXps__1VLfNpzAakQ' \
  -H 'Content-Type: application/json' \
  -d '{
    "nombre": "Laptop ASUS",
    "precio": 2500000,
    "categoria": "Computadores",
    "proveedor": "Distribuciones Metrio"
  }'
```

Request URL

```
http://127.0.0.1:8000/productos
```

Server response

Code	Details
201	Response body <pre>{ "mensaje": "Producto creado", "producto": { "id": 4, "nombre": "Laptop ASUS", "precio": 2500000, "categoria": "Computadores", "proveedor": "Distribuciones Metrio" }, "creado_por": "admin" }</pre>

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/productos' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "nombre": "Nevera",
    "precio": 2500000,
    "categoria": "Computadores",
    "proveedor": "Distribuciones Metrio"
  }'
```

Request URL

http://127.0.0.1:8000/productos

Server response

Code	Details
------	---------

401	Error: Unauthorized
-----	---------------------

Undocumented

Response body

```
{
  "detail": "Not authenticated"
}
```

Response headers

```
content-length: 30
content-type: application/json
date: Thu, 06 Aug 2026 17:27:21 GMT
server: uvicorn
www-authenticate: Bearer
```

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8000/productos/1' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJqb2VsIiwiaXNwIjoxNzg2MDM5MDgyfQ.oi1IJFz62buQ0r8Xxrj5GBko0ovKY8U-0Gfv0SD_3Vvw'
```

Request URL

http://127.0.0.1:8000/productos/1

Server response

Code	Details
------	---------

403	Error: Forbidden
-----	------------------

Undocumented

Response body

```
{
  "detail": "Requiere rol de administrador"
}
```

Curl

```
curl -X 'POST' \  
  'http://127.0.0.1:8000/auth/registro' \  
  -H 'accept: application/json' \  
  -H 'Content-Type: application/json' \  
  -d '{  
    "username": "Jose",  
    "password": "Jose123",  
    "nombre": "Jose"  
  }'
```

Request URL

http://127.0.0.1:8000/auth/registro

Server response

Code

Details

201

Response body

```
{  
  "mensaje": "Usuario registrado correctamente",  
  "username": "Jose",  
  "rol": "cliente"  
}
```

6. Evidencia 4 — Transferencia (reto)

Código de la evidencia	Ej.: GA2-AA3-EV04
Tipo	Producto

Enlace al repositorio con el reto resuelto:

<https://github.com/usuario/mi-repositorio>

<https://github.com/JBUR10/APIS-FASTAPIS> (Carpeta GUIA_3)

Explique brevemente su solución:

¿Qué endpoints implementó? ¿Qué le costó más? ¿Cómo lo probó?

Realicé nuevamente el proceso que ya se realizó con productos, por lo que fue medianamente fácil, el código era fácil de entender incluso sin saber como funciona internamente el sistema de hasheos de Bscrypt, lo probe usando Swagger y realicé un par de pruebas para verificar el funcionamiento correcto de las mismas.

Capturas del reto funcionando:

↓ Pegue aquí las capturas del reto probado en /docs (incluya un 404 y un 422)

404

Undocumented

Error: Not Found

Response body

```
{  
  "detail": "Producto no encontrado"  
}
```


422

Error: Unprocessable Content

Response body

```
{
  "detail": [
    {
      "type": "json_invalid",
      "loc": [
        "body",
        46
      ],
      "msg": "JSON decode error",
      "input": {},
      "ctx": {
        "error": "Illegal trailing comma before end of object"
      }
    }
  ]
}
```

Response headers

```
content-length: 153
content-type: application/json
date: Thu, 06 Aug 2026 17:44:28 GMT
server: uvicorn
```

```

@router.post("", status_code=201)
def crear_categoria(
    datos: CategoriaEntrada,
    usuario: dict = Depends(seguridad.obtener_usuario_actual)
):
    nuevo_id = max([c["id"] for c in categorias], default=0) + 1

    nueva = {
        "id": nuevo_id,
        "nombre": datos.nombre
    }

    categorias.append(nueva)

    return {
        "mensaje": "Categoria creada",
        "categoria": nueva,
        "creado_por": usuario["username"]
    }

@router.put("/{categoria_id}")
def actualizar_categoria(
    categoria_id: int,
    datos: CategoriaEntrada,
    usuario: dict = Depends(seguridad.obtener_usuario_actual)
):
    for categoria in categorias:
        if categoria["id"] == categoria_id:
            categoria["nombre"] = datos.nombre

            return {
                "mensaje": "Categoria actualizada",
                "categoria": categoria,
                "actualizado_por": usuario["username"]
            }

    raise HTTPException(
        status_code=404,
        detail="Categoria no encontrada"
    )

@router.delete("/{categoria_id}")
def eliminar_categoria(
    categoria_id: int,
    admin: dict = Depends(seguridad.requerir_admin)
):
    for categoria in categorias:
        if categoria["id"] == categoria_id:
            categorias.remove(categoria)

            return {
                "mensaje": "Categoria eliminada",
                "categoria": categoria,
                "eliminado_por": admin["username"]
            }

    raise HTTPException(
        status_code=404,
        detail="Categoria no encontrada"
    )

```

Request URL

http://127.0.0.1:8000/categorias

Server response

Code

Details

401

Undocumented

Error: Unauthorized

Response body

```

{
  "detail": "Not authenticated"
}

```

```
http://127.0.0.1:8000/categorias/1
```

Server response

Code	Details
403	Error: Forbidden

Undocumented

Response body

```
{
  "detail": "Requiere rol de administrador"
}
```

Response headers

```
content-length: 42
content-type: application/json
date: Thu, 06 Aug 2026 17:41:20 GMT
server: uvicorn
```

Request URL

```
http://127.0.0.1:8000/categorias
```

Server response

Code	Details
201	

Response body

```
{
  "mensaje": "Categoria creada",
  "categoria": {
    "id": 4,
    "nombre": "Juguetes bebés"
  },
  "creado_por": "joel"
}
```

Response headers

```
content-length: 98
content-type: application/json
date: Thu, 06 Aug 2026 17:41:53 GMT
server: uvicorn
```

7. Lista de chequeo (autoevaluación)

Antes de subir a Classroom, verifique que cumplió con todo. Marque cada casilla:

- ✓ Diligencíe mis datos (nombre, documento, ficha, número de guía).
- ✓ Respondí la Evidencia 1 — Reflexión inicial.
- ✓ Entregué la Evidencia 2 — Cuadro/mapa de contextualización.
- ✓ Entregué la Evidencia 3 — Laboratorio, con capturas de /docs.
- ✓ Entregué la Evidencia 4 — Reto, con enlace a GitHub y capturas.
- ✓ Mi repositorio de GitHub es público y el enlace funciona.
- ✓ Incluí capturas de un error 404 y un error 422 donde se pidió.
- ✓ Exporté el documento a PDF antes de subirlo.